Uploading Images to Cloudinary REST API backend in React JS

Emanuela Sarema
Sep 24, 2020

So I would be writing about a very challenging task I faced when building a social media app for a client application. I was building a social media app for the client application, and I was using Cloudinary as the cloud storage for uploading an .MP3. So the issue here was that Cloudinary is for hosting apps that are mainly in production, and I was building a social media app for a client application, and I was using Cloudinary as the cloud storage.

Cloudinary is a cloud service that offers you a unified backend for your media-rich applications, a flexible set of APIs, and a rich set of libraries and integrations for your preferred development environments. I get it there, and I had been in touch with Cloudinary support.

AND ALL HAPPENS

The process here was simple, given that I was because I do my backend configurations, I had to create a Cloudinary account, and I had to get the credentials from the Cloudinary dashboard.

Tools and Frameworks

- **Cloudinary**: Cloudinary is a cloud service that offers you a unified backend for your media-rich applications, a flexible set of APIs, and a rich set of libraries and integrations for your preferred development environments.
- **Express**: Express is a minimal and flexible Node.js web application framework that provides a robust set of features for building web applications, and I had to use it for my backend.
- **Multer**: Multer is a middleware for handling multipart/form-data, which is primarily used for uploading files. It is written on top of Express.
- **Axios**: Axios is a promise-based HTTP client for the browser and node.js. I had to use it for my frontend.
- **Vue.js**: Vue.js is a progressive, incrementally-adaptable, JavaScript framework for building UI on the web. I had to use it for my frontend.
- **Vuex**: Vuex is a state management pattern + library for Vue.js, serving as a centralized store for all the components in an application, with rules ensuring that the state can only be mutated in a predictable fashion. I had to use it for my frontend.
- **Vuetify**: Vuetify is a Material Design component framework for Vue.js. I had to use it for my frontend.
- **Vue-Axios**: Vue-Axios is a Vue.js integration for Axios. I had to use it for my frontend.
- **Vue-Cli**: Vue-Cli is a simple command line interface for rapid Vue.js development. I had to use it for my frontend.
- **Dotenv**: Dotenv is a zero-dependency module that loads environment variables from a .env file into process.env. I had to use it for my backend.
- **Cors**: Cors is a node.js package for providing Connect/Express middleware that can be used to enable CORS with various options. I had to use it for my backend.
- **Nodemon**: Nodemon is a tool that helps develop node.js based applications by automatically restarting the node application when file changes in the directory are detected. I had to use it for my backend.
- **Vue-Cloudinary-Upload**: Vue-Cloudinary-Upload is a Cloudinary upload component for Vue.js. I had to use it for my frontend.
- **Cloudinary**: Cloudinary is a node.js module to interface with the Cloudinary API. I had to use it for my backend.
- **Vue-Cli-Plugin-Vuetify**: Vue-Cli-Plugin-Vuetify is a Vuetify plugin for Vue-Cli. I had to use it for my frontend.
- **Uuid**: Uuid is a RFC4122 (v1, v4, and v5) UUID generator. I had to use it for my backend.

Project Outlook

Above are the connected libs I will be using for my Uploader.

The project will be divided into two directories - frontend and backend.

So, let us start with the frontend first, and create the root project folder:

```
mkdir cloud-uploader && cd cloud-uploader
```

Now, we will create the folders `client` and `server` which will contain the client and the server.

```
mkdir client server
```

Now, we will create the folders `client` and `server` which will contain the client and the server.

```
vue create client
```

Now, we will create the folders `client` and `server` which will contain the client and the server.

```
cd server && npm init -y
```

This command creates a package directory and an entry file in the current and prints the new package.ts. Alternatively, Nodemon's `init` flag `-y` will create a package.ts file.

Setting Started

Let's start with the backend.

As with every Express project, we need to set up a minimal environment, using Express and Node.js. Express is a fast, unopinionated, minimalist web framework for Node.js. Express is the standard server framework for Node.js.

```
npm i -S express
```

Now we create our server `app.ts` in the root of our project folder, `server`. We will import the Express and Cors packages, and create an instance of Express, and we will use the Express middleware `express.urlencoded()` to parse incoming requests with URL-encoded payloads, and we will use the Express middleware `express.ts()` to parse incoming requests with JSON payloads, and we will use the Express middleware `cors()` to enable CORS with various options, and we will use the Express middleware `express.static()` to serve static files, and we will use the Express middleware `express.static()` to serve static files, and we will use the Express middleware `express.static()` to serve static files